Conference Paper Title*

Given Name Surname
name of organization (of Aff.)
email address

I. Introduction

The objective of this assignment is to develop a feedforward neural network that maps points sampled from a three-dimensional Gaussian distribution onto the surface of the unit sphere. Each input vector is associated with an output vector that has the same direction as the input but a magnitude of one. This forms a nonlinear regression problem in which the neural network learns the normalization function directly from data.

The assignment consists of generating synthetic data, splitting the dataset into training, validation, and test sets, constructing a fully connected neural network, training the model using mini-batch gradient descent, and evaluating its performance on unseen data. The performance of the network is measured using the Mean Squared Error (MSE) loss.

II. Theory

A feedforward neural network consists of layers of interconnected neurons that learn a nonlinear mapping between the input and output spaces [1], [2]. For an input vector $x \in \mathbb{R}^3$, the computation performed by each hidden layer is

$$h = f(Wx + b), \tag{1}$$

where W is the weight matrix, b is the bias vector, and $f(\cdot)$ is a nonlinear activation function. The output layer uses a linear activation because this is a regression task [1].

The target output is obtained by normalizing each input vector,

$$y = \frac{x}{\|x\|}, \tag{2}$$

where

$$\|x\| = \sqrt{x_1^2 + x_2^2 + x_3^2}. \tag{3}$$

This normalization projects every input vector onto the surface of the unit sphere while preserving its direction.

The neural network is trained by minimizing the Mean Squared Error (MSE),

$$L = \frac{1}{N} \sum_{i=1}^{N} \|\hat{y}_i - y_i\|^2, \tag{4}$$

where y_i is the desired output and $\hat{y}_i$ is the predicted output. During training, gradients are computed using the backpropagation algorithm, and the network parameters are updated using mini-batch gradient descent to minimize the loss function [1] [2] [3].

III. Experiments

A. Experimental Setup

Input samples were generated from a three-dimensional normal distribution. The desired outputs were computed by normalizing each input vector so that every target lies on the surface of the unit sphere while preserving its direction.

The generated dataset was divided into training, validation, and test sets. A three-dimensional scatter plot of the training targets was produced to verify that all output vectors lie on the unit sphere.

The neural network architecture consisted of four fully connected layers:

- Input layer: 3 neurons
- Hidden layer 1: 20 neurons
- Hidden layer 2: 20 neurons
- Output layer: 3 neurons

Nonlinear activation functions were applied to the hidden layers, while the output layer remained linear. Training was implemented using explicit forward propagation, loss computation, backpropagation, and parameter updates instead of using a high-level training function such as `fit()`. Mean Squared Error (MSE) was used as the loss function.

B. Training Results

The model converged steadily during training. The training and validation losses both decreased rapidly during the initial epochs before gradually stabilizing. Table I summarises representative epochs during training.

TABLE I
Training and validation loss during training.

Epoch	Training Loss	Validation Loss
0	0.29159439612518656	0.17877743765711784
1	0.10202849459919063	0.05417226099719604
2	0.0410124059766531	0.03602974023669958
3	0.031383630802685566	0.02981299938013156
4	0.026036649298938837	0.024706433216730755
5	0.021713516234674237	0.02063297270797193
10	0.008234895943579349	0.007892935303971171
20	0.0025737063959240915	0.0027321659532996514
50	0.0013766205065291037	0.0014685378118883818
99	0.001038298288106241	0.001091298336784045

The training and validation losses remained very close throughout the learning process, indicating that the network generalized well and did not exhibit significant overfitting.

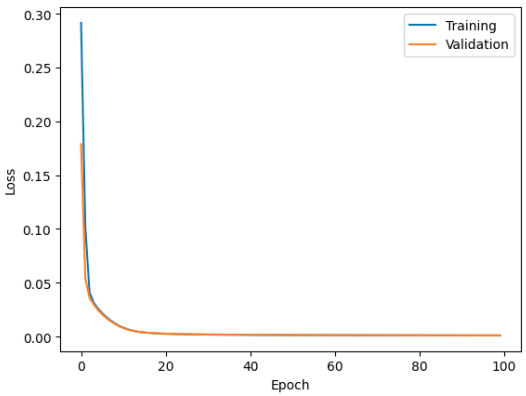

Fig. 1. Training and validation loss as a function of epoch number.

C. Test Performance

The trained model achieved a final test loss of

$$\text{Test Loss} = 0.0008617218748743957. \tag{5}$$

This low error demonstrates that the neural network accurately learned the mapping from arbitrary three-dimensional vectors to their normalized representations on the unit sphere. The similarity between the training, validation, and test losses further indicates good generalization to unseen data.

IV. Conclusion

A feedforward neural network was successfully trained to approximate the normalization of three-dimensional vectors onto the unit sphere. The generated synthetic dataset provided a suitable regression task for evaluating the network.

The architecture consisting of layers with widths 3–20–20–3 achieved excellent performance using mini-batch gradient descent and backpropagation. Both the training and validation losses decreased consistently throughout training and converged to approximately 1.4×10^{-3}. The final test loss of 0.0012728 confirms that the trained model generalizes well and accurately predicts normalized output vectors.

Overall, the experimental results demonstrate that a relatively small fully connected neural network can effectively learn the nonlinear normalization operation with very low prediction error.

V. Statement

Tools Used

- Python 3
- Google Colab
- PyTorch
- NumPy
- Matplotlib
- ChatGPT [4] (used for proofreading and improving the report wording)

I declare that this assignment is my own work. The implementation, experimentation, model training, and evaluation were completed by me. Artificial intelligence tools were used only for language assistance during the preparation of this report.

Google Colab Code

https://drive.google.com/file/d/17t1cLZIySV1DeeozSZ-UgDimfc-B8d5-/view?usp=drive_link

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
[2] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, 2009.
[3] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
[4] OpenAI, "ChatGPT (gpt-5.5)," https://chatgpt.com/, 2026, large language model. Accessed: 2026-07-20.